



Instituto Infnet

Coleta e visualização de dados

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 2 · Etapa 2

Prof. Renan Santos

Onde paramos (Aula 1)

Planejamos o projeto e montamos um painel — mas com amostra manual.

- **Problema afiado + método (CRISP-DM/TDSP)**

Charter, Data Summary, estrutura de pastas.

A pergunta: quais UFs priorizar em saneamento?

- **App Streamlit com AMOSTRA de 8 UFs**

Dados colados à mão num CSV.

Funcionava — mas não escala e desatualiza.

- **Decisão da Avaliação: iterar**

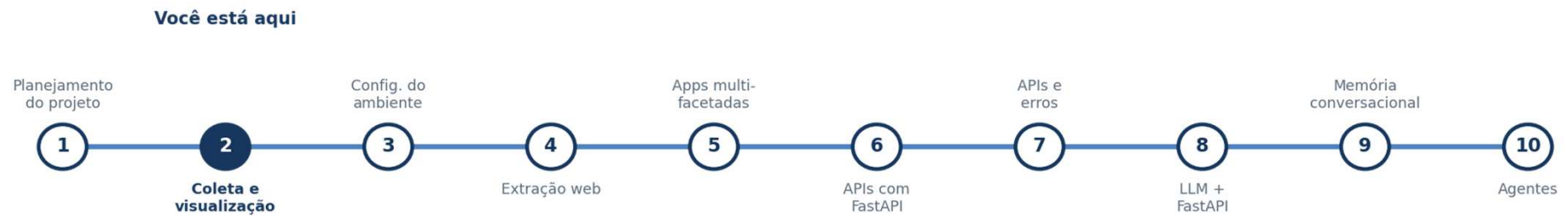
Automatizar a coleta e cobrir as 27 UFs.

É o que faremos hoje (a v2).

► **A v1 provou o conceito; a v2 o torna real. Isso é o TDSP iterando.**

Onde estamos no Projeto de Bloco

Etapa 2: coleta e visualização. Cada etapa acrescenta uma peça ao MESMO projeto.



Da amostra à coleta real

 *A iteração do nosso caso: o painel passa a se atualizar sozinho.*

- **v1 (Aula 1): amostra manual**

8 UFs digitadas num CSV.

Frágil: erra, desatualiza, não escala.

- **v2 (hoje): coleta via API + gráfico**

27 UFs coletadas automaticamente do IBGE.

Com filtro, métricas e visualização.

► O mesmo projeto, um degrau acima — sem recomeçar do zero.

Por que não copiar os números à mão?

Objetivos da aula

Cinco capacidades — coleta real + visualização (Competência 1).

- **Escolher fontes/tipos de dados**

Arquivo × API × WebScraping — e quando usar cada.

- **Coletar via API REST em Python**

requests + json, tratando status e erros.

- **Transformar JSON em tabela**

JSON aninhado → lista de dicionários; tratar tipos.

- **Visualizar em Streamlit**

Gráfico, filtro por região, métricas.

- **Robustez + separação de camadas**

Cache/fallback; dados isolados da interface.

O entregável: TP1

A Aula 2 fecha a parte técnica do TP1.

- **Coleta via API + visualização**
Exatamente o que construímos hoje.
- **Falta plugar o SEU indicador/ODS**
Sobre a base de hoje, com o seu tema.
- **Ao fim de hoje: ~90% do TP1**
Problema (Aula 1) + coleta + gráfico (Aula 2).

► Depois de hoje, o TP1 é sobretudo adaptar ao seu problema.



1 • De onde vêm os dados?

Recap: tipos de dado

O tipo decide como tratamos — e a API do IBGE devolve um deles.

- **Estruturado**

Tabela/CSV/SQL — linhas e colunas. Ótimo para análise.

- **Semi-estruturado**

JSON, XML — típico de APIs. É o que o IBGE devolve.

- **Não-estruturado**

Texto, áudio, imagem — sem esquema fixo.

► Hoje: JSON (semi) → tabela (lista de dicionários) para visualizar.

Três formas de obter dados

Arquivo, API ou WebScraping — cada um tem seu momento.

- **Arquivo (CSV/JSON)**

Baixar um arquivo pronto. Bom para dado estático/bulk.
Mas: desatualiza; download manual.

- **API**

Pedir dado a um sistema via HTTP. Atualizado e integrável.
Mas: às vezes paga/limitada [Esppenchutz, p75].

- **WebScraping**

Extrair de páginas HTML — quando NÃO há API.
Frágil (o HTML muda); ver termos de uso.

► **No caso: o IBGE tem API pública e gratuita → é a melhor escolha.**

O que é uma API REST

Um contrato para dois sistemas trocarem dados por HTTP.

- **Endpoint (URL)**

ONDE pedir. Ex.: /api/v1/localidades/estados.

- **Método GET**

LER dados (POST/PUT/DELETE escrevem — hoje só lemos).

- **Parâmetros**

Refinam o pedido: ?orderBy=nome, filtros...

- **Resposta: JSON + status**

200 = ok · 404 = não achado · 500 = erro do servidor.

► **REST = pedir por URL (GET) e receber JSON com um código de status.**

Anatomia de uma chamada

Seu código faz um GET ao endpoint com parâmetros; o servidor responde 200 + JSON.

Anatomia de uma chamada REST



Autenticação: chave, OAuth e o IBGE

Muitas APIs exigem identificar quem chama — o IBGE não.

- **Chave de API (API key)**

Enviada em params ou header.

Regra: NUNCA hardcode a chave no script [Esppenchutz, p70].

- **OAuth / Bearer**

Fluxo mais robusto para dados sensíveis.

Ex.: APIs do Google [Esppenchutz, p73].

- **A API do IBGE é aberta**

Sem chave — simplifica hoje.

Mas guarde o alerta: volta com LLMs pagas.

▶ Aberta hoje; mas 'não hardcode segredo' é lei para o resto do bloco.



2 · Coleta via API em Python

A chamada básica (requests + json)

src/data_access.py

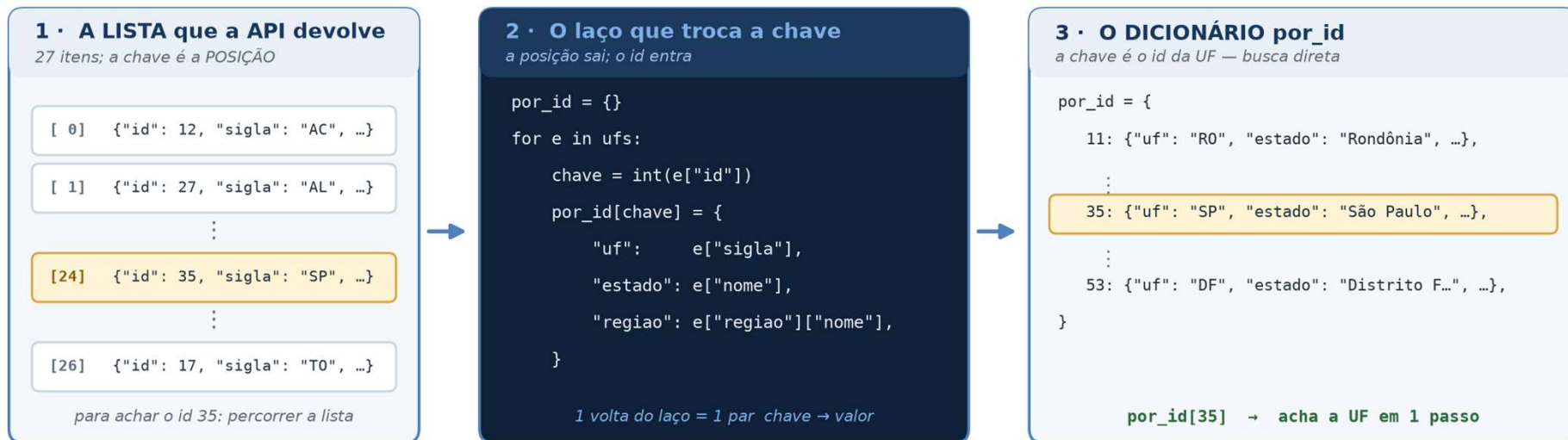
```
import requests

url = "https://servicodados.ibge.gov.br/api/v1/localidades/estados?orderBy=nome"
resp = requests.get(url, timeout=20)
resp.raise_for_status()      # erro se status != 2xx (tratamento de erro!)
estados = resp.json()        # JSON -> lista de dicionários
```

Da lista ao dicionário: por_id

A API devolve uma LISTA; viramos um DICIONÁRIO por id — a chave que junta os dados depois.

A mesma informação em duas estruturas — e por que a segunda ganha



Cuidado com o TIPO: em /localidades o id vem número (35); em /agregados vem TEXTO ("35"). Convertendo os dois com `int(...)`, a chave 35 encontra a UF no Passo 4.

O JSON aninhado, camada por camada

A resposta não é uma tabela: é caixa dentro de caixa. Cada acesso abre UMA camada até as series.

Por que aquela linha tem tantos colchetes

Cada acesso abre UMA camada

cada colchete do caminho destranca a caixa seguinte

`resp.json()` lista com 1 elemento

`[0]` dict: id · variavel · unidade · resultados

`["resultados"]` lista com 1 elemento

`[0]` dict: classificacoes (vazio) + series

`["series"]` ★ LISTA com 27 itens — 1 por UF

Dentro de UMA série (series[0])

onde moram os dois tropeços de tipo

```
{  
  "localidade": {  
    "id": "35",  
    "nome": "São Paulo"  
  },  
  "serie": {"2025": "46081801"}  
}
```

TEXTO!

TEXTO!

os dois vêm como texto → int(...) no Passo 4

a linha inteira: `series = resp.json()[0]["resultados"][0]["series"]` → 27 séries, 1 por UF

Navegar o JSON aninhado (semi → tabela)

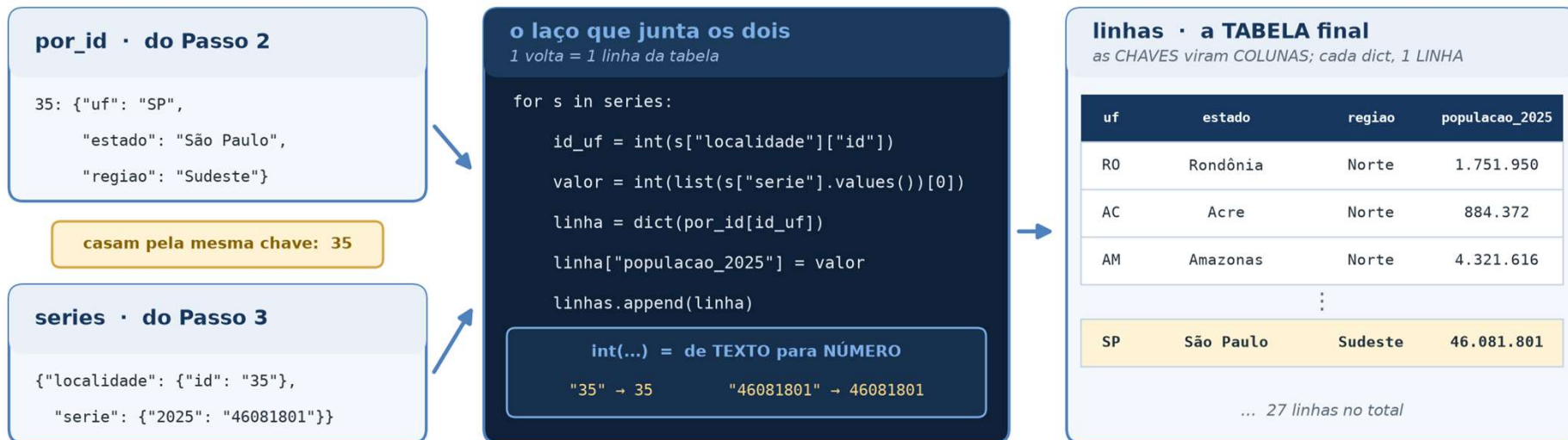
src/data_access.py

```
series = resp_pop.json()[0]["resultados"][0]["series"]
linhas = []
for s in series:
    id_uf = int(s["localidade"]["id"])          # id vem como TEXTO -> int
    valor = int(list(s["serie"].values())[0])    # {"2025": "46081801"} -> valor
    linha = dict(por_id[id_uf]); linha["populacao_2025"] = valor
    linhas.append(linha)
```

Do JSON à tabela: as 27 linhas

por_id + series casam pela mesma chave; cada volta do laço produz UMA linha da tabela.

Duas estruturas entram, uma tabela sai



linhas é uma LISTA de 27 dicionários — e é exatamente isso que `st.dataframe(linhas)` e `st.bar_chart(linhas, x="uf", y="populacao_2025")` consomem. Sem pandas.

Isolar a camada de dados

A coleta mora em `src/data_access.py` — longe da interface.

- **Separação de camadas (SoC)**

Dados em `src/`; interface em `app.py`.

Se o IBGE mudar o endpoint, mexe-se só no acesso.

- **Interface simples: `carregar_dados()`**

Devolve (linhas, fonte).

O app não sabe COMO os dados vêm — só pede.

► Trocar a fonte não deve quebrar a tela — por isso separamos.

Anti-padrões de API

Os erros clássicos ao consumir uma API — e como evitá-los.

- **Hardcode de chave/URL espalhado**

Centralize; nunca versione segredos [Esppenschutz, p70].

- **Sem tratamento de erro**

Um 500 vira dado corrompido silencioso. Use `raise_for_status/try`.

- **'Martelar' a API a cada clique**

Use cache (a seguir). Respeite limites.

- **Misturar dados e interface**

Separe em `src/`. E leia a doc (paginação!).

► **Código de coleta bom é defensivo: trata erro, não repete, e fica isolado.**



Case v2 — os tropeços de hoje

Construir de verdade sempre revela surpresas — de novo.

- **JSON aninhado**

A resposta não era tabela; navegamos `[0]['resultados'][0]['series']`.

- **Tipos, de novo**

id como texto ('35') → `int(...)` para casar com os estados.

- **E se a API cair na aula?**

Decisão: cache/fallback (próximo bloco).

► O dado real exige código defensivo — é para isso que as fases de dados existem.



3 · Visualização e robustez

Cache: não recoletar a cada clique

app.py

```
import streamlit as st
from src.data_access import carregar_dados

@st.cache_data(ttl=3600)    # guarda o resultado por 1 hora
def obter_dados():
    return carregar_dados()

dados, fonte = obter_dados()
```

Filtro por região (st.multiselect)

app.py

```
regioes = sorted({l["regiao"] for l in dados})
selecionadas = st.multiselect("Filtrar por região", regioes, default=regioes)
filtrados = [l for l in dados if l["regiao"] in selecionadas]
```

Gráfico: população por UF (st.bar_chart)

app.py

```
ordenados = sorted(filtrados, key=lambda l: l['populacao_2025'], reverse=True)
st.bar_chart(ordenados, x="uf", y="populacao_2025")
```

```
# st.bar_chart aceita a LISTA DE DICIONÁRIOS (x=/y=) -> sem pandas
```

Robustez: fail gracefully

Um app não deve morrer porque a rede oscilou.

- **carregar_dados() tenta a API...**
...e, se falhar, usa o cache (ufs_cache.json).
E avisa a fonte na tela.
- **Por que isso importa**
Rede de sala é traiçoeira; produção também cai.
Degradar com elegância > erro vermelho na frente da turma.

► **Fallback é a diferença entre 'quebrou' e 'está usando o cache'.**

Temos o painel v2!

27 UFs coletadas ao vivo, filtro por região, métricas e gráfico — tudo em Python puro.

Painel de Indicadores Sustentáveis do Brasil

Coleta ao vivo via API de dados abertos do IBGE

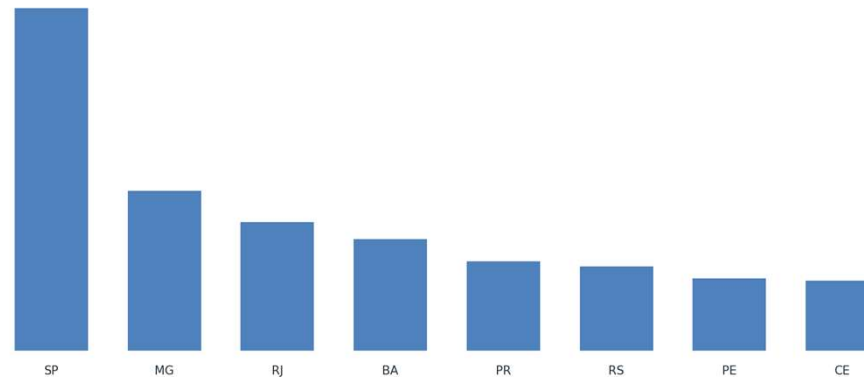
Fonte: API do IBGE (ao vivo) · 27 UFs | Filtrar por região: [Todas]

UFs exibidas
27

Regiões
5

População somada
213.421.037

População por UF



Princípios que levamos

Três ideias que valem para qualquer coleta de dados.

- **Separação de camadas**

Dados (src/) $\neq$ interface (app.py).
Troca a fonte sem quebrar a tela.

- **Fail gracefully**

Cache/fallback: o app sobrevive à rede.
Sempre tenha um plano B para o dado.

- **Leituras cacheáveis (GET)**

GET não muda o servidor $\rightarrow$ pode cachear.
DRY: uma função de coleta, reutilizada.

► **Coletar bem é tanto sobre robustez quanto sobre obter o dado.**

O caso, em 3 lições (v2)

O que a iteração do Painel ODS nos ensinou.

- **API > copiar à mão**

Escala (27, 5.570 municípios), atualiza, não erra.

- **Código defensivo**

Status, tipos, JSON aninhado — o dado real surpreende.

- **Robustez**

Cache + fallback funcionam na sala e em produção.

► **Método + código defensivo + robustez = um painel que você confia.**

Para a próxima etapa

E quando a fonte NÃO tiver API?

- **WebScraping (Etapa 4)**

Extrair de páginas HTML quando não há API/arquivo.

- **Seu TP1 agora**

Plugue o seu indicador/ODS sobre a base de hoje.

► O caso continua: v3 = saneamento (ODS 6) e fontes sem API.

Em 3 frases

O mapa mental da Etapa 2.

- **Escolha a fonte certa**

Arquivo × API × scraping — hoje, API do IBGE.

- **Colete com código defensivo**

requests+json, status, tipos, JSON aninhado, camada isolada.

- **Visualize com robustez**

Streamlit: filtro + gráfico; cache + fallback.

► **Da pergunta (Aula 1) ao dado real na tela (Aula 3).**